



UNIVERSIDAD DE SANTANDER — UDES

Ingeniería de Sistemas — Electiva Aplicaciones Móviles



Fresh Eats

Aplicación Móvil Android para Gestión de Inventario de Alimentos y
Sugerencia de Recetas

Documento:	Informe de Proyecto Final — 2026
Autores:	Sebastián Cano
	Miguel Ángel Rizo Arias
Docente:	Jhonatan Rolando Rey Castillo
Universidad:	Universidad de Santander (UDES)
Sede:	Cúcuta, Norte de Santander, Colombia
Repositorio:	https://github.com/Sebxxs004/FreshEats

Documento presentado como requisito del proyecto final — Cúcuta, 2026



Resumen

Fresh Eats es una aplicación móvil para Android construida en Kotlin con Jetpack Compose. La idea es simple: el usuario registra los ingredientes que tiene en casa y la app le muestra qué puede cocinar con ellos. Si decide preparar algo para lo que le falta un ingrediente, la app lo calcula y se lo indica con exactitud.

Para hacerlo funcionar se combinaron tres cosas: la arquitectura MVVM que Google recomienda para Android, Firebase para manejar los datos del usuario en la nube, y la API de Spoonacular como fuente de recetas e información de ingredientes. El resultado es una app que responde en tiempo real a lo que el usuario tiene disponible en su despensa.

Palabras clave: Android, Kotlin, Jetpack Compose, MVVM, Firebase, Spoonacular, inventario de alimentos, recetas, desperdicio alimentario.

Abstract

Fresh Eats is a native Android app built with Kotlin and Jetpack Compose. The idea: users log what ingredients they have at home, and the app tells them what they can cook. If they want a recipe but are missing something, the app calculates exactly what's needed.

The stack combines MVVM architecture, Firebase for cloud persistence, and the Spoonacular API as the recipe engine. The app updates in real time as the user's inventory changes.

Keywords: Android, Kotlin, Jetpack Compose, MVVM, Firebase, Spoonacular, food inventory, recipes, food waste.



Tabla de Contenido

1.	Introducción	4
2.	El Problema	4
3.	Por Qué Vale la Pena Hacerlo	5
4.	Objetivos	5
5.	Marco Conceptual	6
6.	Cómo Funciona el Sistema	7
7.	Arquitectura del Proyecto	9
8.	Tecnologías Usadas	9
9.	Firebase en Detalle	10
10.	La API de Spoonacular	11
11.	El Motor de Comparación de Ingredientes	11
12.	Diseño de la Interfaz	12
13.	Lo Que Aprendimos	13
14.	Qué Vendría Después	13
15.	Referencias	14



1. Introducción

Hay una situación cotidiana que casi todos conocen: abrir el refrigerador, ver un montón de ingredientes que llevan días ahí y no saber bien qué hacer con ellos. Al final, parte de eso termina en la basura. No por descuido, sino porque no siempre es fácil conectar lo que uno tiene con lo que puede cocinar.

Fresh Eats nació de esa observación. La app no le dice al usuario qué comprar ni le impone un menú semanal — le muestra qué puede cocinar hoy, con lo que ya tiene. Y si quiere preparar algo para lo que le falta un ingrediente, la app hace el cálculo y se lo indica.

Este documento describe el proceso de diseño y desarrollo del proyecto: el problema que se quería resolver, las decisiones técnicas que se tomaron, cómo quedó la arquitectura y qué aprendimos en el camino.

2. El Problema

Según la FAO (2019), alrededor de un tercio de los alimentos producidos en el mundo se pierde o desperdicia. En los hogares, una causa frecuente es la falta de planificación: se compran ingredientes sin un plan claro de uso y terminan venciendo antes de que alguien los aproveche.

Las apps de recetas existentes resuelven solo una parte del problema. Le muestran al usuario qué cocinar, pero asumen que ya tiene todo lo necesario. Si no es así, el usuario tiene que revisar por su cuenta si le falta algo. Eso genera fricción y, en la práctica, mucha gente simplemente no lo hace.

El enfoque de Fresh Eats es al revés: primero el usuario registra lo que tiene, y después la app le dice qué puede cocinar. Si quiere algo para lo que le falta un ingrediente, la app calcula exactamente cuánto le falta de cada cosa. La idea es que la decisión de qué preparar parta del inventario real, no de una lista ideal.

3. Por Qué Vale la Pena Hacerlo

Reducir el desperdicio de alimentos tiene impacto real. A nivel ambiental, produce menos residuos orgánicos. A nivel económico, las familias gastan menos en lo que no van a usar. Y a nivel práctico, ayuda a planificar mejor las compras.



Desde el lado técnico, el proyecto era una buena oportunidad para aplicar herramientas modernas de Android — Jetpack Compose, Firebase, Kotlin Coroutines — en un contexto concreto y con lógica propia, no solo como ejercicio de sintaxis. La integración de una API externa como Spoonacular añadía un componente interesante: datos reales, con todo lo que eso implica en términos de manejo de errores, tiempos de respuesta y experiencia de usuario.

4. Objetivos

Objetivo general

Desarrollar una aplicación móvil Android que permita a los usuarios gestionar su inventario de ingredientes y recibir sugerencias de recetas basadas en lo que tienen disponible, con cálculo automático de ingredientes faltantes, usando Firebase y la API de Spoonacular.

Objetivos específicos

- Implementar autenticación con Firebase Authentication (correo y contraseña).
- Construir un inventario de ingredientes por usuario sincronizado con Cloud Firestore.
- Integrar el autocompletado de ingredientes y la búsqueda de recetas de Spoonacular.
- Desarrollar la lógica que calcula la diferencia entre lo que pide una receta y lo que tiene el usuario.
- Diseñar la interfaz con Material Design 3, siguiendo criterios de accesibilidad.
- Estructurar el código con MVVM usando Coroutines y StateFlow para el manejo de estado.

5. Marco Conceptual

Kotlin

Kotlin es el lenguaje oficial de Android desde 2017. Tiene null safety integrado, es más conciso que Java y tiene soporte nativo para corrutinas, que son la forma que se usó en este proyecto para manejar operaciones asíncronas como llamadas a la red y lectura de base de datos (Google, 2017; JetBrains, 2023).

Jetpack Compose

Compose es el sistema de UI declarativo de Android, estable desde 2021. En lugar de XML, la interfaz se describe como funciones de Kotlin que reaccionan al estado. Cuando el estado cambia, Compose vuelve a calcular solo las partes afectadas de la pantalla. Eso simplifica mucho el manejo del ciclo de vida de la UI (Google, 2021).

Arquitectura MVVM

MVVM separa la lógica de presentación (ViewModel) de la interfaz (View) y del acceso a datos (Model). Google la recomienda para Android porque facilita las pruebas y hace el código más fácil de mantener. El ViewModel expone el estado como StateFlow; los composables lo observan y se actualizan automáticamente (Google Android Developers, 2023).

Firebase

Firebase es la plataforma de backend de Google para apps móviles. En este proyecto se usaron dos servicios: Authentication para el manejo de sesiones, y Cloud Firestore como base de datos NoSQL con sincronización en tiempo real. Firestore permite escuchar cambios en los documentos sin hacer polling — algo que fue clave para que el inventario del usuario se actualice al instante en todas las pantallas (Google Firebase, 2023).

API REST y Spoonacular

Una API REST es una interfaz que permite a dos sistemas comunicarse por HTTP (Fielding, 2000). Spoonacular es una API gastronómica con más de 360.000 recetas, información nutricional, autocompletado de ingredientes y búsqueda de recetas por ingredientes disponibles. En Fresh Eats, es el motor que transforma el inventario del usuario en sugerencias concretas de qué cocinar (Spoonacular, 2023).

6. Cómo Funciona el Sistema

La app no tiene backend propio. Toda la lógica del servidor corre en Firebase y Spoonacular. El cliente Android se comunica con esos dos servicios y combina sus respuestas.

Módulo	Qué hace	Tecnología
Autenticación	Registro e inicio de sesión	Firebase Auth
Inventario	Ingredientes del usuario en la nube	Cloud Firestore + Spoonacular
Recetas	Sugerencias basadas en el inventario	Spoonacular API
Detalle de Receta	Instrucciones, nutrición y diferencial	Spoonacular + Firestore
Favoritos	Guardar recetas preferidas	Cloud Firestore
Navegación	Flujo entre pantallas	Jetpack Navigation Compose

Tabla 1. Módulos principales de Fresh Eats.

Autenticación

El flujo de login y registro usa Firebase Authentication con correo y contraseña. La sesión queda persistida: si el usuario cierra la app y la vuelve a abrir, no necesita iniciar sesión de nuevo. El estado de autenticación vive en un ViewModel compartido que expone un StateFlow — cualquier cambio se propaga automáticamente a las pantallas que lo estén observando.

Inventario de ingredientes

Cada usuario tiene una subcolección inventory dentro de su documento en Firestore. Cada documento dentro de esa subcolección representa un ingrediente: nombre, imagen (obtenida de Spoonacular), cantidad y unidad. Para agregar ingredientes, el usuario escribe en un campo de búsqueda con autocompletado que consulta la API con un debounce de 300 ms — así no se dispara una petición por cada tecla presionada. Solo se guardan ingredientes reconocidos por Spoonacular, lo que evita datos inconsistentes.

Sugerencia de recetas

La pantalla principal lee el inventario del usuario y hace una petición al endpoint /recipes/findByIngredients de Spoonacular, pasando los ingredientes como lista. Los resultados son recetas ordenadas por compatibilidad con el stock del usuario. Las



recetas marcadas como favoritas se guardan en Firestore y están disponibles sin necesidad de otra llamada a la API.

Detalle de una receta

Al abrir una receta, se consulta `/recipes/{id}/information` para obtener los pasos de preparación, la lista de ingredientes con cantidades exactas y la información nutricional. Al mismo tiempo se descarga el inventario del usuario para hacer la comparación. El resultado de esa comparación es lo que se muestra en pantalla: qué ingredientes ya tiene y cuánto le falta de los que no tiene.



7. Arquitectura del Proyecto

La estructura sigue el patrón MVVM con tres capas. Las dependencias van siempre hacia abajo — la UI no sabe nada del repositorio, y el repositorio no sabe nada de la UI.

Capa	Qué contiene	Responsabilidad
UI / View	Composables de Jetpack Compose	Mostrar estado, capturar eventos del usuario
ViewModel	ViewModel por cada pantalla	Exponer StateFlow, manejar lógica de presentación
Repository	Repositorios de datos	Conectar con Firestore y con Retrofit
Data Source	Firebase SDK + Retrofit/OkHttp	Hacer las llamadas reales a los servicios

Tabla 2. Capas de la arquitectura MVVM en Fresh Eats.

Los repositorios convierten las respuestas de Firestore en Flows de Kotlin usando callbackFlow. Los ViewModels combinan esos Flows con operadores como combine y map, y los exponen como StateFlow. Los composables los observan con collectAsStateWithLifecycle() y se recomponen solos cuando el estado cambia. Los eventos del usuario (tocar un botón, escribir en un campo) van siempre hacia el ViewModel — la UI nunca toca datos directamente.

8. Tecnologías Usadas

Tecnología	Versión	Para qué se usa
Kotlin	1.9+	Lenguaje principal
Jetpack Compose	BOM 2024	Construcción de la interfaz
Coroutines & Flow	1.7+	Operaciones asíncronas y estado reactivo
Retrofit + OkHttp	2.11 / 4.x	Llamadas a la API de Spoonacular
Coil Compose	2.x	Carga de imágenes desde URL
Navigation Compose	2.7+	Navegación entre pantallas
Firebase Auth	22+	Autenticación de usuarios
Cloud Firestore	24+	Base de datos en la nube
Material Design 3	Compose M3	Sistema de diseño visual
Spoonacular API	REST v1	Recetas e información de ingredientes



Android Studio	Hedgehog+	IDE de desarrollo
Gradle KTS	8.x	Sistema de build del proyecto

Tabla 3. Stack tecnológico de Fresh Eats.



9. Firebase en Detalle

Authentication

Se implementó registro e inicio de sesión con correo y contraseña. Firebase maneja el hash de contraseñas y la seguridad de tokens — no se almacena ninguna contraseña en el dispositivo ni en Firestore. La sesión persiste entre reinicios de la app.

Cloud Firestore — Estructura de datos

La base de datos está organizada alrededor del usuario. Cada usuario tiene un documento en la colección users, con dos subcolecciones:

- **inventory/{itemId}**: los ingredientes del usuario (nombre, imagen, cantidad, unidad).
- **favorites/{recipeId}**: los IDs de las recetas que marcó como favoritas.

Los repositorios usan SnapshotListeners de Firestore, convertidos a callbackFlow. Eso significa que cualquier cambio en la base de datos llega a la app sin que ella tenga que pedirlo — Firestore empuja el cambio, la app lo recibe, el Flow lo propaga y Compose actualiza la pantalla. Todo ocurre en milisegundos.

10. La API de Spoonacular

/food/ingredients/autocomplete

Se usa para el campo de búsqueda del inventario. Devuelve sugerencias de ingredientes reales con sus imágenes oficiales. El debounce de 300 ms evita que se haga una petición por cada carácter que el usuario escribe.

/recipes/findByIngredients

Recibe la lista de ingredientes del usuario y devuelve recetas ordenadas por compatibilidad. Las primeras en la lista son las que el usuario puede cocinar con lo que ya tiene; las siguientes requieren algún ingrediente adicional.

/recipes/{id}/information

Devuelve el detalle completo de una receta: instrucciones paso a paso, lista de ingredientes con cantidades en varias unidades, y datos nutricionales incluyendo calorías.

11. El Motor de Comparación de Ingredientes

Esta es la parte más interesante del proyecto desde el punto de vista técnico. Cuando el usuario abre el detalle de una receta, el `RecipeDetailViewModel` hace dos cosas al mismo tiempo: consulta los ingredientes requeridos a Spoonacular y descarga el inventario actual del usuario desde Firestore.

Esas dos fuentes de datos se combinan con el operador `combine` de Kotlin Flow. Cada vez que alguna de las dos cambia — por ejemplo, si el usuario agrega un ingrediente al inventario mientras tiene abierta la pantalla de una receta — el cálculo se ejecuta de nuevo automáticamente.

El resultado es directo: si una receta pide 5 tomates y el usuario tiene 2, la pantalla muestra que le faltan 3. Los ingredientes que ya tiene se marcan en verde; los que le faltan, en naranja con la cantidad exacta.

12. Diseño de la Interfaz

La interfaz sigue Material Design 3 con una paleta propia basada en tonos verdes y naranja. Los verdes transmiten frescura y salud; el naranja se reserva para advertencias y para indicar ingredientes faltantes.

Color	Hex	Dónde se usa
Green Primary	#2E7D32	Elementos principales, encabezados
Green Light	#4CAF50	Confirmaciones, ingredientes disponibles
Orange Primary	#E65100	Advertencias, ingredientes faltantes
Green Surface	#E8F5E9	Fondos de tarjetas y listas

Tabla 4. Paleta de colores de Fresh Eats.

Para las unidades de medida del inventario se usaron Exposed Dropdown Menus de Material 3, lo que obliga al usuario a elegir de una lista predefinida (gramos, tazas, litros, unidades) en lugar de escribir libremente. Eso evita inconsistencias cuando se compara con las cantidades que devuelve Spoonacular.

Las imágenes de ingredientes y recetas se cargan con Coil, que maneja el caché automáticamente y muestra placeholders mientras la imagen llega. Los contrastes de color cumplen con el nivel AA de las pautas WCAG 2.1.



13. Lo Que Aprendimos

El proyecto funcionó. La app hace lo que se propuso: toma el inventario del usuario y le dice qué puede cocinar. Y cuando no tiene todo, le dice exactamente qué le falta.

Lo más valioso del proceso fue trabajar con Kotlin Flow en un contexto real. La combinación de callbackFlow con Firestore y combine para cruzar datos de dos fuentes resultó más limpia de lo esperado — una vez que el patrón estaba claro, el código se escribió casi solo.

La API de Spoonacular trajo sus propias complicaciones. Los datos no siempre llegan en las unidades que uno esperaría, y normalizar cantidades para comparar inventario con recetas requirió más trabajo del planificado. Es el tipo de problema que solo aparece cuando se trabaja con datos reales.

Una conclusión práctica: no tener backend propio simplificó mucho el despliegue, pero también puso límites claros. Si en el futuro se quisiera agregar lógica más compleja — recomendaciones personalizadas, por ejemplo — habría que pensar si Firebase solo alcanza o si conviene agregar un servidor propio.

14. Qué Vendría Después

- Notificaciones cuando un ingrediente del inventario está próximo a vencer, usando Firebase Cloud Messaging.
- Reconocimiento de ingredientes por foto con Firebase ML Kit — el usuario apunta la cámara y la app lo agrega al inventario.
- Planificación de menú semanal con lista de compras consolidada.
- Versión para iOS usando Kotlin Multiplatform Mobile.
- Métricas de desperdicio evitado para que el usuario vea el impacto concreto de usar la app.

15. Referencias

Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Tesis doctoral]. Universidad de California, Irvine.



- Food and Agriculture Organization of the United Nations. (2019). *The state of food and agriculture 2019: Moving forward on food loss and waste reduction*. FAO. <https://doi.org/10.4060/ca6030en>
- Google. (2017). *Kotlin is now an official language for Android development*. Android Developers Blog. <https://android-developers.googleblog.com/2017/05/android-announces-support-for-kotlin.html>
- Google. (2021). *Jetpack Compose is now stable*. Android Developers Blog. <https://android-developers.googleblog.com/2021/07/jetpack-compose-announcement.html>
- Google Android Developers. (2023). *Guide to app architecture*. Android Developers. <https://developer.android.com/topic/architecture>
- Google Firebase. (2023). *Cloud Firestore documentation*. Firebase. <https://firebase.google.com/docs/firestore>
- Google Firebase. (2023). *Firebase Authentication documentation*. Firebase. <https://firebase.google.com/docs/auth>
- JetBrains. (2023). *Kotlin coroutines guide*. Kotlin Documentation. <https://kotlinlang.org/docs/coroutines-guide.html>
- Ministerio de Ambiente y Desarrollo Sostenible de Colombia. (2018). *Política nacional para la gestión sostenible del suelo*. MADS. <https://www.minambiente.gov.co>
- Spoonacular. (2023). *Spoonacular food API documentation*. <https://spoonacular.com/food-api/docs>

Repositorio: <https://github.com/Sebxxs004/FreshEats> · Cúcuta, 2026